

TD/TP 3 DE CALCUL NUMÉRIQUE

Résolution de système linéaire

T. Dufaud (thomas.dufaud@uvsq.fr)

—M1 CHPS—

Cette fiche de TD/TP est associée au cours de calcul numérique sur l'algèbre linéaire. Elle a pour objectifs :

- Découvrir un langage haut niveau, type DSL, pour le calcul scientifique.
- Savoir implémenter un algorithme pour le calcul numérique.
- Comprendre les performances et les limites d'un algorithmes.
- Comparer les performances.
- **Résoudre des systèmes linéaires.**

L'exercice 1 (TD) est à réaliser sur feuille. Les exercices 2, 3 et 4 (TP) sont à réaliser sur machine avec Scilab. **Un compte rendu doit être réalisé.** Ce compte rendu doit comprendre l'analyse des algorithmes implémentés, vos observations et analyses d'algorithme, ainsi que des mesures de performance. Il complète le compte rendu du TD/TP 2.

► Exercice 1. [TD - Matrice triangulaire et méthode directe]

On souhaite résoudre le système $Ax = b$, où A est une matrice triangulaire inférieure de taille $n \times n$ à coefficients réels, $x, b \in \mathbb{R}^n$.

- Ecrivez la procédure de résolution directe par la méthode de descente.
- Quelle est la complexité de cet algorithme ?
- Quelle est l'occupation mémoire de votre procédure ?

► Exercice 2. TP - Système triangulaire

1. Ecrivez les algorithmes 10 et 11 (diapo 22) de résolution par remontée et descente. On crée 2 fichiers `usolve.sci` et `lsolve.sci` avec les en-têtes :

```
function [x] = usolve(U,b)
```

et

```
function [x] = lsolve(L,b)
```

2. Tester et valider vos algorithmes (cf TD/TP 1). On peut s'aider des fonctions `tril()` et `triu()` qui permettent d'extraire une matrice triangulaire inférieure ou supérieure.

► **Exercice 3. TP - Gauss**

1. Ecrivez l'algorithme 12 (diapo 27) de résolution par élimination de Gauss sans pivotage. On crée 1 fichier `gausskij3b.sci`

```
function [x] = gausskij3b(A,b)
```

2. Testez et validez votre algorithme sur de petites matrices.

► **Exercice 4. TP - LU**

1. Ecrire l'algorithme 13 (diapo 30) de factorisation LU

```
function [L,U] = mylu3b(A)
```

2. Testez et validez votre algorithme sur de petites matrices. On calculera l'erreur commise sur la factorisation LU : $A - LU$.
3. Améliorez l'algorithme 13 (diapo 30) de factorisation LU de sorte à n'obtenir qu'une boucle.
4. Ajoutez la méthode de pivot partiel.

```
function [L,U,P] = mylu(A)
```

5. Comparez avec la fonction `lu()` de Scilab.
6. Appliquez `mylu` et `lu` aux matrices de T. Davis. On effectue la lecture avec la fonction `mmread`. (OPTIONNEL)

► **Exercice 5. Poisson 1D (OPTIONNEL)**

1. Écrire une matrice A du problème de poisson 1D de taille n (Equation de la chaleur 1D stationnaire).
2. Écrire le second membre et la solution analytique.
3. Résoudre le système pour différentes tailles de A avec LU. Critiquez les résultats en tenant compte du conditionnement de A .
4. Tracer la solution avec `plot()`
5. Résoudre le système pour différentes tailles de A en utilisant une factorisation de Cholesky. On utilisera la fonction `chol()`.
6. Ecrire un algorithme LU pour une matrice tridiagonale. Comparez vos performances avec les algorithmes précédents.